
OSR TRNG

RO 电路实现指导

文档信息

版本	v1.2
交付客户	
关键字	TRNG， RO 电路实现
摘要	该文档主要描述 OSR TRNG 中 RO 电路实现指导

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2025 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有©2014-2025 深圳市组创信安科技发展有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市组创信安科技发展有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市组创信安科技发展有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

历史版本

版本号	日期	描述
1.0	2022/08/31	初版
1.1	2023/11/24	1. 修改表格 1-1 TRNG 快环目标频率 2. 重新排版
1.2	2023/12/11	增加 RO 测试信号说明

目录

RO 电路实现指导	1
1.1 RO 快环熵源布局	6
1.1.1 布局指导	6
1.1.2 电路延迟要求	6
1.1.3 布局案例	7
1.1.4 版图参考	8
1.2 RO 慢环熵源布局	9
1.2.1 布局指导	9
1.2.2 电路延迟要求	10
1.2.3 布局案例	10
1.2.4 RO_CLK 异步分频器	11
1.3 RO 快环、慢环电路频率关系	12
1.4 熵源环路级数调整	13
1.5 RO 及 RO_CLK 环路间距	15
1.5.1 环路间距离要求	15
1.5.2 版图参考案例	15
1.6 环路的物理集成	19
1.7 后端实现检查指导	20
1.7.1 RO 环路实现频率确认	20
1.7.2 RO 环路版图确认	20
1.7.3 RO 环路摆放位置调整	21
2 TRNG RO 测试信号说明	22
3 TRNG 启动时间说明	23

TRNG 的 RO 电路包括 4 组，每组包括 16 个快环熵源电路和 1 个慢环采样电路。其结构是慢环采样时钟经过分频后去采样快环熵源时钟。

示意图如下：

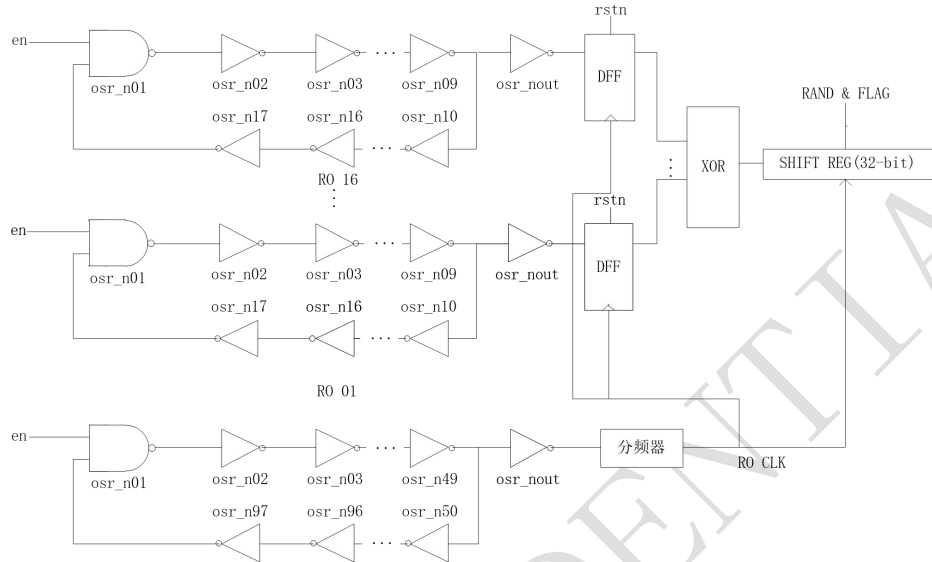


图 1-1 单路随机源示意图

本章节内容为以 UMC55 工艺举例，通过改变熵源环路中反相器个数的方法来控制环路实现的频率。章节中的反相器个数和目标频率（600M 与 100M）和分布规律为举例和参考作用，实际实现时由于工艺、工艺角、实现方法的不同，结果也会有一定差异，不需与文档描述完全一致。需保证的是环路间频率差异足够大，以提供足够的随机性，和防止环路间的共振影响。

1.1 RO 快环熵源布局

HDL 文件名:

- ro_circuit.v

模块名:

- ro_circuit

例化名前缀:

- OSR_DONTTOUCH_

电路图:

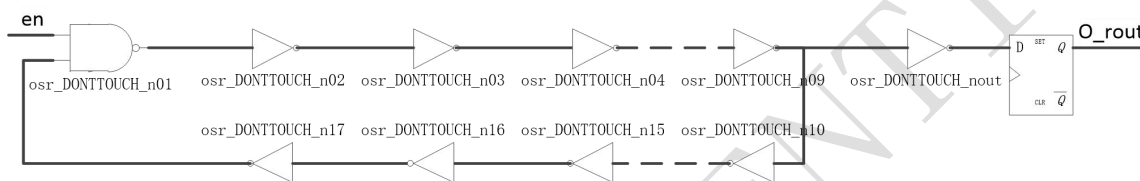


图 1-2 OSR RO CIRCUIT SCHEMATIC

1.1.1 布局指导

RO 快环电路包含一个与非门和一些反相器以及一个寄存器。这里建议按照上图所示的摆放顺序对电路中的逻辑器件进行摆放，每个 RO 环在数字版图中只放置在两行内。

假设一共有 17 个反相器组合 RO 环，那么第一行包含 osr_DONTTOUCH_n01 到 osr_DONTTOUCH_n09 和 osr_DONTTOUCH_nout，剩下的反相器放置在第二行，按照相反的顺序排列。上下两行的标准单元在垂直方向对齐，例如，osr_DONTTOUCH_n02 应和 osr_DONTTOUCH_n17 垂直方向保持一致。

1.1.2 电路延迟要求

在 Typical case 下，为达到 600MHz 振荡频率，RO 快环环路的延迟要求为 830ps。

也就是说，从 osr_DONTTOUCH_n01 输出端，至 osr_DONTTOUCH_n09 输出端，再到 osr_DONTTOUCH_n10 输入端，一直返回到 osr_DONTTOUCH_n01 的输入端；整个环路的延迟为要求 830ps。期望震荡频率为 600MHz。

以 UMC55 工艺举例，INV_X1 器件在 TT 下的延时为 49ps， $830/49=16.9 \approx 17$ 个反相器。

其他工艺下请按照对应工艺计算 $830/\text{INV_X1_delay}$ 得出相邻最近的奇数个反相器数量，或者以 OSR 交付为基准在反相器之间插入 buffer 的方式调整环路延时。

占空比要求为 40%~60%。

设计中有 4 个熵源，每个熵源内含有 16 个 RO 快环，即一共 64 个快环环路。

All rights reserved.

这里建议每个熵源的 16 个 RO 快环电路延时具有略微差异来降低熵源环路之间的共振概率，例如：

方案 1： 如果环路器件数量较少，可以通过逐次将 X1 器件替换成 X2 器件来引入环路间的差异，环路 1（17 个 INV_X1）；环路 2（16 个 INV_X1 + 1 个 INV_X2）；环路 3（15 个 INV_X1 + 2 个 INV_X2）以此类推.....

方案 2： 如果环路上器件较多，可以依次多插入一个 buffer 来引入环路间的差异。

1.1.3 布局案例

下表为 TRNG 熵源 RO 快环目标频率案例（四组熵源目标一致）

表格 1-1 TRNG 快环目标频率案例

RO 快环	目标频率/MHz	误差/MHz
ro_01	530	+/-5MHz
ro_02	540	+/-5MHz
ro_03	550	+/-5MHz
ro_04	560	+/-5MHz
ro_05	570	+/-5MHz
ro_06	580	+/-5MHz
ro_07	590	+/-5MHz
ro_08	600	+/-5MHz
ro_09	610	+/-5MHz
ro_10	620	+/-5MHz
ro_11	630	+/-5MHz
ro_12	640	+/-5MHz
ro_13	650	+/-5MHz
ro_14	660	+/-5MHz
ro_15	670	+/-5MHz
ro_16	680	+/-5MHz

注：表格中的频率做参考和举例用，实际实现时在保证环路之间频率有足够的差异的基础上，频率数值或分布规律均可与表格有差异。

1.2 RO 慢环熵源布局

HDL 文件名:

- ro_clk_circuit.v

模块名:

- ro_clk_circuit

例化名前缀:

- OSR_DONTTOUCH_

电路图:

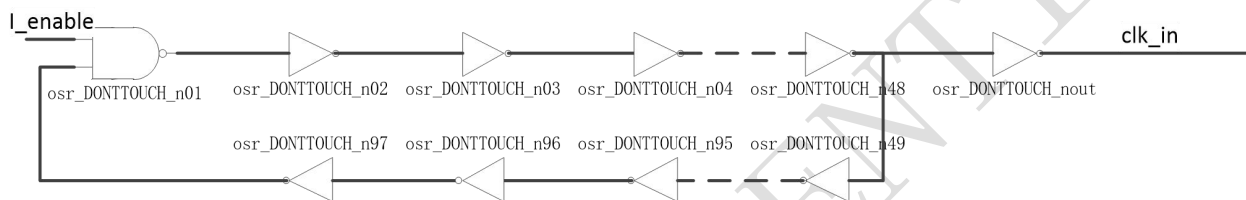


图 1-4 OSR RO CLK CIRCUIT SCHEMATIC

1.2.1 布局指导

RO 慢环电路结构与 RO 快环电路结构类似。在 UMC55 工艺下其环路中包含 1 个二输入与非门及 96 个反相器。

由于 RO 慢环环路较长，若按照 RO 快环的方式进行摆放的话，可能造成其版图过于细长，不利于后续布局。因此，建议 RO 慢环采用弯折的方式用多行形成环路。

例如：上述 1 个与非门加上 96 个反相器构成的链路，可以采用 6 行的方式放置器件。

第一行包含 osr_DONTTOUCH_n01 到 osr_DONTTOUCH_n17 和 osr_DONTTOUCH_nout。

第二行放置 osr_DONTTOUCH_n18 到 osr_DONTTOUCH_n33。其中 osr_DONTTOUCH_n18 与 osr_DONTTOUCH_n17 对齐，osr_DONTTOUCH_n33 与 osr_DONTTOUCH_n02 对齐。

上下两行的器件方向相反。

第三行放置 osr_DONTTOUCH_n34 到 osr_DONTTOUCH_n49。其中 osr_DONTTOUCH_n34 与 osr_DONTTOUCH_n33 对齐，osr_DONTTOUCH_n49 与 osr_DONTTOUCH_n18 对齐。

上下两行的器件方向相反。

第四行放置 osr_DONTTOUCH_n50 到 osr_DONTTOUCH_n65。其中 osr_DONTTOUCH_n50 与 osr_DONTTOUCH_n49 对齐，osr_DONTTOUCH_n65 与 osr_DONTTOUCH_n34 对齐。

上下两行的器件方向相反。

第五行放置 `osr_DONTTOUCH_n66` 到 `osr_DONTTOUCH_n81`。其中 `osr_DONTTOUCH_n66` 与 `osr_DONTTOUCH_n65` 对齐，`osr_DONTTOUCH_n81` 与 `osr_DONTTOUCH_n50` 对齐。上下两行的器件方向相反。

第六行放置 `osr_DONTTOUCH_n82` 到 `osr_DONTTOUCH_n97`。其中 `osr_DONTTOUCH_n82` 与 `osr_DONTTOUCH_n81` 对齐，`osr_DONTTOUCH_n97` 与 `osr_DONTTOUCH_n66` 对齐。上下两行的器件方向相反。

走线由 `osr_DONTTOUCH_n01` 从左往右到 `osr_DONTTOUCH_n17`，向下到 `osr_DONTTOUCH_n18`，从右往左到 `osr_DONTTOUCH_n33`，向下到 `osr_DONTTOUCH_n34`，从左往右到 `osr_DONTTOUCH_n49`，如此往复直到 `osr_DONTTOUCH_n97`；最后从 `osr_DONTTOUCH_n97` 环回到 `osr_DONTTOUCH_n01`，构成环路。`osr_DONTTOUCH_nout` 则由 `osr_DONTTOUCH_n48` 输出驱动。

RO 熵源部分共有 4 路 RO 慢环，建议在 P&R 时分散摆放在数字版图中。

1.2.2 电路延迟要求

在 Typical case 下，RO 慢环环路的延迟要求为 5ns。

也就是说，从 `osr_DONTTOUCH_n01` 输出端，至 `osr_DONTTOUCH_n97` 输出端，一直返回到 `osr_DONTTOUCH_n01` 的输入端；整个环路的延迟为 5ns。期望震荡频率为 100MHz。

与 RO 快环电路类似，可以在对应的工艺下计算得到需要的器件数量，不过更建议是在环路中插 buffer 的方式调整环路延时。

占空比要求为 40%~60%。

设计中有 4 个熵源，每个熵源内含有 1 个 RO 慢环，即一共 4 个慢环环路。4 个环路也建议引入一些额外的差异来提升随机性设计，建议依次多插入一个 buffer 来引入环路间的差异。

1.2.3 布局案例

表格 1-2 TRNG 慢环目标频率案例

RO 慢环	目标频率/MHz	误差/MHz
ros1	100	+/-2.5MHz
ros2	105	+/-2.5MHz
ros3	110	+/-2.5MHz
ros4	115	+/-2.5MHz

注：表格中的频率做参考和举例用，实际实现时在保证环路之间频率有足够的差异的基础上，频率数值或分布规律均可与表格有差异。

1.2.4 RO_CLK 异步分频器

对于 `ro_clk` 异步分频器，在 P&R 时不需要手工干预，但需要与 RO 慢环环路放在一起。

OSR CONFIDENTIAL

1.3 RO 快环、慢环电路频率关系

在保证 RO 快环、慢环电路频率大约 6:1 关系并确保电路正常工作的前提下，可以根据实际需求 and 工艺提高或降低快、慢环频率。

OSR CONFIDENTIAL

1.4 熵源环路级数调整

TRNG RO 环级数为参数可调，下图列出了如何调整快环、慢环的级数。

- 调整 RO 快环电路级数

RTL: rtl/crypto_plain/trng/ro*/ro_source.v

```
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 0), INV_N(17), .delay(810))ro_01(.en(i_roen[ 0]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 0]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 1), INV_N(23), .delay(830))ro_02(.en(i_roen[ 1]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 1]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 2), INV_N(19), .delay(570))ro_03(.en(i_roen[ 2]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 2]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 3), INV_N(25), .delay(870))ro_04(.en(i_roen[ 3]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 3]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 4), INV_N(21), .delay(890))ro_05(.en(i_roen[ 4]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 4]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 5), INV_N(17), .delay(710))ro_06(.en(i_roen[ 5]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 5]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 6), INV_N(23), .delay(730))ro_07(.en(i_roen[ 6]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 6]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 7), INV_N(19), .delay(590))ro_08(.en(i_roen[ 7]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 7]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 8), INV_N(25), .delay(770))ro_09(.en(i_roen[ 8]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 8]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+ 9), INV_N(21), .delay(790))ro_10(.en(i_roen[ 9]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[ 9]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+10), INV_N(17), .delay(610))ro_11(.en(i_roen[10]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[10]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+11), INV_N(23), .delay(630))ro_12(.en(i_roen[11]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[11]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+12), INV_N(19), .delay(670))ro_13(.en(i_roen[12]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[12]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+13), INV_N(25), .delay(690))ro_14(.en(i_roen[13]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[13]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+14), INV_N(21), .delay(510))ro_15(.en(i_roen[14]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[14]));
osr_trng_ro_circuit #(.p_IDX(16*p_IDX+15), INV_N(17), .delay(530))ro_16(.en(i_roen[15]),.clk(w_clk),.rstn(w_rst_n),.scan(i_scan),.o_rout(w_rbito[15]));
```

图 1-5 RO 快环级数调整

- 调整 RO 慢环电路级数

RTL: rtl/crypto_plain/trng/ro*/ro_top.v

```
osr_trng_ro_source #(.p_IDX(0), .CLK_INV_N(97), .delay(5010))
ros1(
    .clk      (clk),
    .resetn   (resetn),
    .ro_rst   (w_ro_rst1),
    .i_scan   (i_scan),
    .i_roen   (w_roen[3]),
    .i_conf   (w_conf),
    .i_roen   (w_roen[63:48]),
    .o_rand   (w_rand1),
    .i_flag   (w_iflag1),
    .o_flag   (w_oflag1),
    .o_rornd  (w_rornd1),
    .o_roclk  (w_roclk1)
);

osr_trng_ro_source #(.p_IDX(1), .CLK_INV_N(95), .delay(4770))
ros2(
    .clk      (clk),
    .resetn   (resetn),
    .ro_rst   (w_ro_rst2),
    .i_scan   (i_scan),
    .i_roen   (w_roen[2]),
    .i_conf   (w_conf),
    .i_roen   (w_roen[47:32]),
    .o_rand   (w_rand2),
    .i_flag   (w_iflag2),
    .o_flag   (w_oflag2),
    .o_rornd  (w_rornd2),
    .o_roclk  (w_roclk2)
);
```

```
osr_trng_ro_source #(.p_IDX(2), .CLK_INV_N(93)) .delay(4510))
ros3(
  .clk      (clk      ),
  .resetn   (resetn   ),
  .ro_rst   (w_ro_rst3 ),
  .i_scan   (i_scan   ),
  .i_rose   (w_rose[1] ),
  .i_conf   (w_conf   ),
  .i_roen   (w_roen[31:16] ),
  .o_rand   (w_rand3  ),
  .i_flag   (w_iflag3 ),
  .o_flag   (w_oflag3 ),
  .o_rornd  (w_rornd3 ),
  .o_roclk  (w_roclk3 )
);

osr_trng_ro_source #(.p_IDX(3), .CLK_INV_N(91)) .delay(4170))
ros4(
  .clk      (clk      ),
  .resetn   (resetn   ),
  .ro_rst   (w_ro_rst4 ),
  .i_scan   (i_scan   ),
  .i_rose   (w_rose[0] ),
  .i_conf   (w_conf   ),
  .i_roen   (w_roen[15: 0] ),
  .o_rand   (w_rand4  ),
  .i_flag   (w_iflag4 ),
  .o_flag   (w_oflag4 ),
  .o_rornd  (w_rornd4 ),
  .o_roclk  (w_roclk4 )
);
```

图 1-6 RO 慢环级数调整

1.5 RO 及 RO_CLK 环路间距

1.5.1 环路间距离要求

由前可知，设计中共有 4 个 RO 慢环及 64 个 RO 快环环路。其中 RO 慢环频率为 100MHz 震荡信号，RO 快环为 600MHz 震荡信号。为了避免同频震荡信号出现互锁，进而影响随机性的问题，要求同频环路之间保持一定距离。

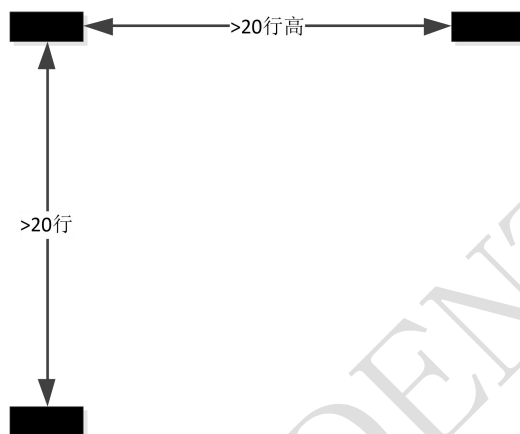


图 1-7 RO CIRCUIT 间距示意

如图 1-7 所示，建议 RO 快环之间横向与纵向之间的距离大于 20 倍的行高。

同样，RO 慢环之间横向与纵向之间的距离大于 200 倍的行高。

1.5.2 版图参考案例

下图提供了一种版图参考，实际布局时可按需调整。

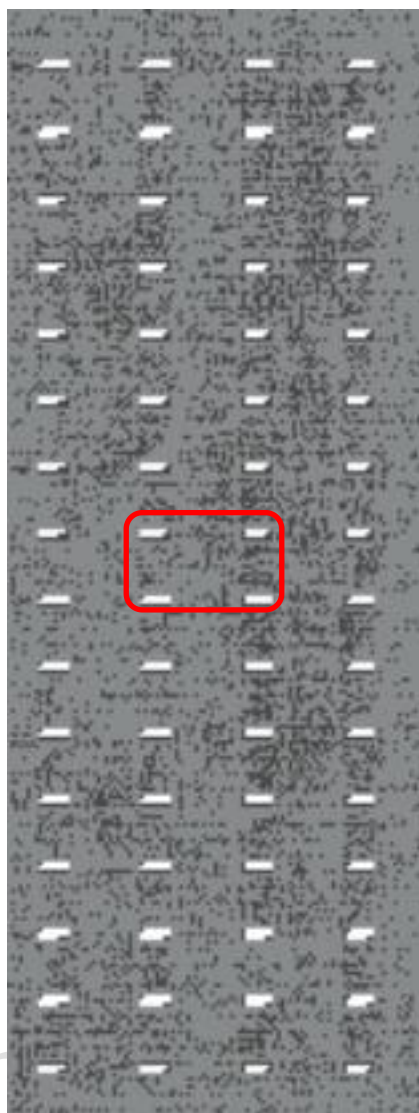


图 1-8 RO 快环摆放参考

上图展示了 64 个 RO 快环的一种摆放方式。对每一路熵源内的 16 个 RO 快环可采用矩阵，线形排列或散落放置。以图中任意四个 RO 快环为例，其间距示意如下图（具体数值无指导意义）。下图中的 RO 快环摆放遵循了章节 1.1.1 中建议：摆放在两行内。

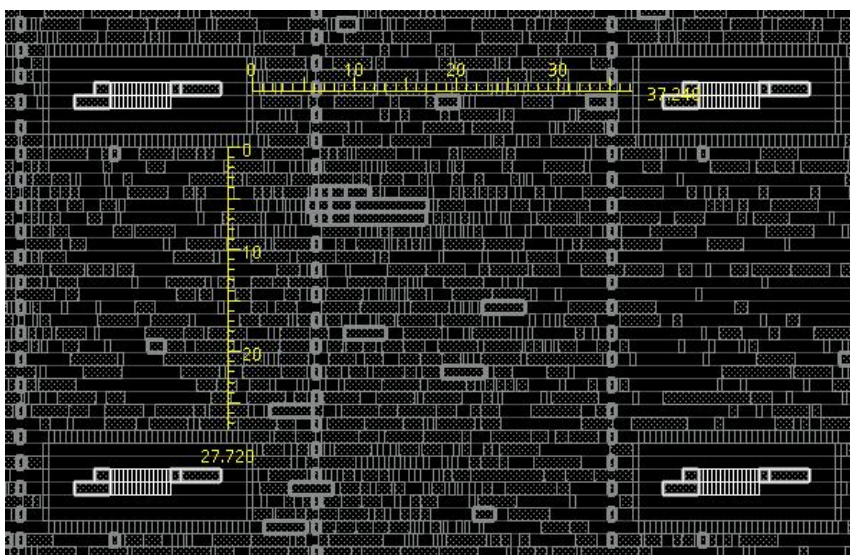


图 1-9 RO 快环间距示意图



图 1-10 RO 慢环摆放参考

上图展示了 4 个 RO 慢环的一种摆放方式，与 RO 快环类似，可按需选择排列方式。其间距示意如下图（具体数值无指导意义）：

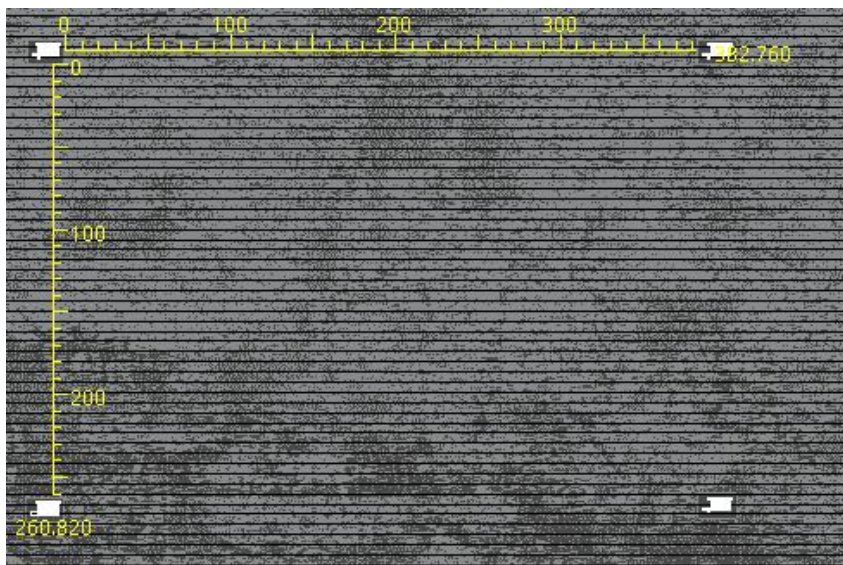


图 1-11 RO 慢环间距示意图

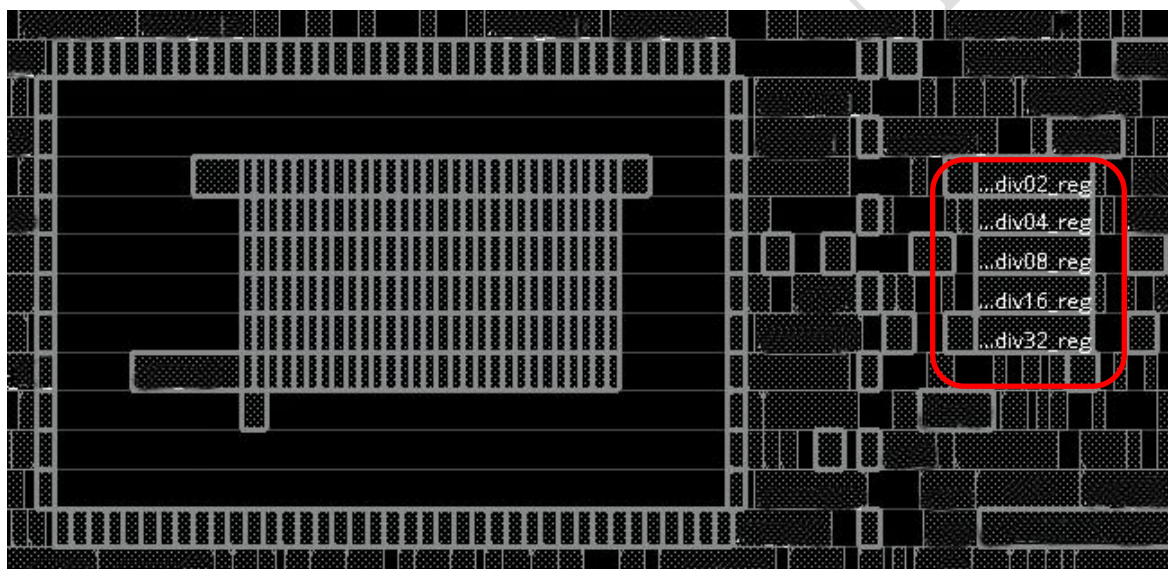


图 1-12 RO_CLK 异步分频器摆放参考

上图展示了 RO 慢环的一种实现参考，并体现了了 1.6 章节中前两条建议的实现方法。同时展示了异步分频器与 RO 慢环环路相对位置的参考。

1.6 环路的物理集成

RO 环路的输出端进行了时序约束,被驱动电路的信号质量可以由后端工具和静态时序分析工具来保证,能够保证涉及到的 **transition**、**noise**、**crosstalk**、**IR drop** 等分析结果干净,相关逻辑的物理设计能够达到 **Sign off** 标准。

而 RO 环路的输出端之前的环路逻辑,无法被时序约束约束到,因此工具无法分析该信号对邻近逻辑的串扰等影响。为了降低 RO 环路本身对其他数字电路的串扰等影响,请尽量保证 RO 环路和相关逻辑的间距。此处有以下建议:

- 在把 RO 环路做成宏单元的时候, 周围加一圈 **decap cell**, 形成占位隔离
- 在绕线的时候,对 RO 环路周围可能被影响到的数字信号酌情进行“双倍”/“三倍”/“四倍”间距及线宽的调整,降低串扰影响
- RO 环路摆放好后,使用工具将环路中未被约束到的 **net** 抓取出来,依照设计的要求,限制在其周围走其他数字逻辑的信号线
- 如果周围有非常关键的信号,则可以额外加 **shielding**; 但请注意,增加 **shielding** 对面积有影响

目前推荐前三种方法。

1.7 后端实现检查指导

章节 7 中已正向地介绍了熵源部分环路的实现方法。由于 TRNG 熵源部分为大量组合逻辑环路组成的随机震荡源，为保证其在芯片中可以正常工作，其在前端实现过程中会有根据实现结果进一步调试的需求。本章节介绍在实现后，根据仿真结果进行反向检查的方法。

检查过程中有以下三个问题需要关注：

- 1) 是否需对 RO 环路内反相器个数进行调整，以使 RO 环路实现的频率符合要求。
- 2) 确保 RO 环路内反相器的摆放方式与 RO 环路的摆放间距满足要求，以保证 RO 环路工作的稳定性。
- 3) 在满足摆放间距的基础上，是否需要调整环路排列的顺序进行特殊调整。

1.7.1 RO 环路实现频率确认

建议优先在 TT 工艺角仿真得出 4 个 RO 慢环和 64 个 RO 快环的环路延时或频率，在其他工艺角下虽然 RO 环实现的频率可能与 TT 工艺角不同，但其频率差异规律大致相同。如有更全面的检查需求可选择进行更详细的 PVT 反馈仿真。

对上述仿真得到的环路延时结果进行如下步骤：

- 1) 将延时信息换算为频率 (/MHz)，需注意 RO 环的延时为二分之一震荡周期。
- 2) 计算每一组 RO 环的平均频率。
- 3) 计算相邻环路频率差异，如 ro_02 与 ro_01 间差异、ro_03 与 ro_02 间差异等。
- 4) 计算步骤 3) 的频率差异占步骤 2) 平均频率的百分比。

1.7.1.1 RO 快环频率确认

至此对于上述 16 个 RO 快环会得到 15 个频率差异百分比。观察是否 15 个百分比中小于 1% 的数量大于 3 个，即有多余 4 组相邻 RO 快环间频率差异小于频率均值的 1%。如有以上频率差异分布则说明相邻环路间整体频率差异较小，需通过调整 RO 快环反相器个数，或插入 buffer 等其他方式，加大全体 RO 快环间频率差异，直到频率差异小于 1% 的环路在四组之内。此步骤提供了一个建议的频率差异下限，如有更高熵源稳定性要求且功耗与资源允许，频率差异均达到 1.6% 左右为最优。

1.7.1.2 RO 慢环频率确认

对于上述 4 个 RO 慢环会得到 3 个频率差异百分比。如有两个或以上频率差异百分比小于 5%，说明相邻环路间整体频率差异较小，需通过调整 RO 慢环反相器个数，或插入 buffer 等其他方式，加大全体 RO 慢环间频率差异，直到至多只有一组相邻环频率差异小于 5%。此步骤提供了一个建议的频率差异下限，如有更高熵源稳定性要求且功耗与资源允许，RO 慢环间频率差异均达到 5% 为最优。

1.7.2 RO 环路版图确认

- 1) 确认每一组 RO 快环与 RO 慢环分别按照 1.1 与 1.2 章节中的布局指导排布。

- 2) 确认环路间的摆放间距符合 1.5 章节的要求，可按 1.5.2 章节中案例中的方法得出摆放间距。
- 3) 确认已根据实际电路结构考虑是否需要加入 16 章节中建议的方法，或通过其他保护方式，保证 RO 环路与其他逻辑电路相对独立。

1.7.3 RO 环路摆放位置调整

下图展示了一个 RO 快环仿真延时结果举例（而非实现目标）：

	ros1	ros2	ros3	ros4
ro_01	0.9249	0.9267	0.9216	0.9199
ro_02	0.9023	0.9068	0.9051	0.9042
ro_03	0.8946	0.8949	0.8956	0.8971
ro_04	0.8775	0.8854	0.8787	0.8808
ro_05	0.8735	0.8699	0.8671	0.8724
ro_06	0.8558	0.8541	0.8553	0.8592
ro_07	0.8492	0.8441	0.8450	0.8479
ro_08	0.8345	0.8330	0.8321	0.8324
ro_09	0.8228	0.8225	0.8210	0.8179
ro_10	0.8065	0.8016	0.8068	0.8094
ro_11	0.7995	0.7980	0.7988	0.7925
ro_12	0.7836	0.7840	0.7831	0.7840
ro_13	0.7739	0.7735	0.7723	0.7726
ro_14	0.7606	0.7527	0.7574	0.7566
ro_15	0.7451	0.7420	0.7441	0.7474
ro_16	0.7339	0.7257	0.7246	0.7303

图 1-13 RO 快环实现频率举例

上图环路延时案例中已满足频率差异小于 1% 的相邻环路小于 4 组，且多数相邻环路间频率差异为 1.6% 左右甚至更高。此时不需要再通过改变环路内器件个数的方式的调整延时。但图中也标出了一组相邻环路间频率差异仍然较小的情况。针对这种有个别相邻环频率差异较小的情况，需要在满足 1.7.2 的要求的环路间距的基础上，调整摆放顺序将上述频率差异较小的环路不相邻摆放。

对于 RO 慢环的摆放顺序的调整方法也与上述 RO 快环相同，将频率差异不足 5% 的环路不相邻摆放。

通过上述调整可使相邻环路的频率差异足够大。但环路的频率仍为递增规律，为了进一步保障相邻环路之间的频率差异，建议将环路整体乱序摆放，并特别关注上述频率差异较小的环路不临近摆放。

2 TRNG RO 测试信号说明

顶层端口 `o_trng_ro_clk` 和 `o_trng_ro_out` 为 OST_TRNG 模块熵源部分测试信号：

表格 2-1 TRNG 测试信号端口

端口名	方向	位宽	描述
<code>o_trng_ro_clk</code>	OUT	4	输出 4 路 RO 熵源内部采样时钟
<code>o_trng_ro_out</code>	OUT	4	测试 4 路 RO 熵源的随机数输出

注：

1. 测试信号均为 4bit 位宽的信号，每个相同位置的信号对应相同的熵源。
顺序为信号[3:0] 分别对应 RO 熵源编号 3-0。
2. 上述测试信号提供了检验熵源是否处于正常工作状态的接口，推荐连接到 GPIO，以便熵源出现问题时观测熵源情况。正常工作情况下，测试信号的每一位应持续震荡，且 `o_trng_ro_clk` 的每一位应有较明显相位差，`o_trng_ro_out` 应呈不规则震荡。
3. 对应熵源的测试信号中，`o_trng_ro_out` 的每一次跳变均发生在 `o_ro_clk` 的上升沿。

下图为测试信号时序图（以 1 路 RO 熵源为例），此两信号实际为 RO 环震荡得到的时钟信号，下图中的周期差异为展示随机误差：

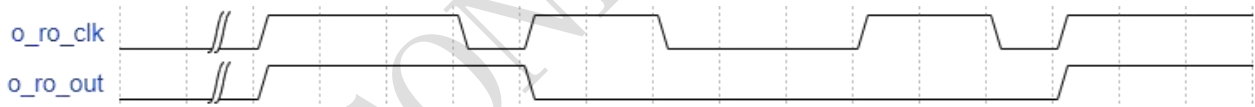


图 2-1 测试信号时序图

3 TRNG 启动时间说明

OSR_TRNG 模块上电并释放复位后，TRNG 将自动进入上电自检过程，上电自检通过后，端口信号 `o_trng_rdy` 将拉高并保持为高电平。上电自检通过时证明 TRBG 输出的随机性符合随机性检测的基本要求，且 DRBG 逻辑电路无异常，此时才会允许随机数输出使用，TRNG 进入功能模式。

由 OSR_TRNG 复位释放到 TRNG 进入功能模式的时间可由以下公式估算：

假设：

TRNG 慢环时钟频率为 F_t ，单位（MHz）

eHSM 主频为 F ，单位（MHz）

TRNG 上电自检期间固定主频时钟周期数： $m=8920$

TRNG 上电自检期间固定慢环时钟周期数： $n=2780$

$$T = m \times \frac{1}{F} + n \times \frac{1}{F_t/32}$$

按以上公式，假设 eHSM 主频时钟为 500MHz，TRNG 慢环时钟频率为 100MHz，采样环分频比为默认的 32 分频，则 TRNG 复位释放后至可输出随机数所需的启动时间为：

$$T = 8920 \times \frac{10^9}{500 \times 10^6} + 2780 \times \frac{10^9}{(100/32) \times 10^6} = 0.907, \text{ 单位 (MS)}$$

章节 2 测试信号中的 `o_trng_ro_clk` 为 RO 熵源内部采样时钟，应以 4 个随机采样时钟信号中最慢的频率代入上述公式计算，因此建议估计频率范围，并代入范围内的最慢频率。

如保留上电自检功能且有较强的缩短启动时间需求，可将采样环分频比设置为 16 分频。此配置将损失一定的随机性。